
CS 301

High-Performance Computing

Assignments 06 & 07 - Parallel Interpolation

with Particle Mover

Prem Kukadiya (202301452)
Suryadeepsinh Gohil (202301463)

April 19, 2026

Contents

1	Lab 6 - Parallel Interpolation	3
1.1	Implementation Approach and Race Conditions (Q1 & Q2)	3
1.1.1	The Race Condition Problem	3
1.1.2	Our Solution: Thread-Local Privatization	3
1.1.3	Pseudocode	3
1.1.4	Implementation Diagram	4
1.2	Hardware Specifications	4
1.3	Performance Analysis & Results	5
1.3.1	Execution Time and Speedup vs Cores (Q3)	5
1.3.2	Parallel Efficiency & Scalability (Q4)	6
1.3.3	Maximum Speedup Achieved (Q5)	6
1.4	Observations and Bottlenecks	6
1.4.1	Explaining the Speedup using HPC Knowledge (Q6)	6
1.4.2	Load Imbalance & Performance Bottlenecks (Q8)	7
1.5	Future Improvements and Alternative Approaches	7
1.5.1	Unlimited Resources Optimization (Q7)	7
1.5.2	Alternative Approaches & Data Structures (Q9 & Q10)	7
1.6	Conclusion	8
2	Lab 7 - Parallel Interpolation with Particle Mover	9
2.1	Implementation Approach & Race Conditions (Q1 & Q2)	9
2.1.1	Parallel Implementation & Handling Race Conditions (Q2)	9
2.1.2	Algorithm Pseudocode (Q1)	9
2.1.3	Implementation Diagram (Q1)	10
2.2	Performance Analysis & Results	11
2.2.1	Speedup and Execution Time vs Cores (Q3 & Q5)	11
2.2.2	Parallel Efficiency & Scalability (Q4)	11
2.2.3	Phase Analysis: Interpolation vs. Mover (Q11)	12
2.3	Observations and Bottlenecks	12
2.3.1	Explaining the Speedup using HPC Knowledge (Q6)	12
2.3.2	Load Imbalance & Performance Bottlenecks (Q8)	13
2.4	Future Improvements & Alternative Approaches	13
2.4.1	Unlimited Resources Optimization (Q7)	13
2.4.2	Alternative Approaches & Parallelization Methods (Q9)	13
2.4.3	Implemented Data Structures (Q10)	13
2.5	Conclusion	14

1 Lab 6 - Parallel Interpolation

Abstract

In this lab, we focus on parallelizing a scatter point-to-mesh interpolation algorithm using OpenMP. Mapping millions of randomly scattered points onto a structured grid in parallel introduces severe race conditions. Instead of using slow atomic locks, we implement a highly efficient "thread-local privatization" strategy. We analyze the performance across five different grid sizes and particle counts on both a Lab PC and an HPC Cluster, evaluating execution time, speedup, and parallel efficiency as we scale from 1 up to 16 cores (specifically 1, 2, 4, 6, 8, 10, 12, 14, and 16) to comprehensively track scaling behavior.

1.1 Implementation Approach and Race Conditions (Q1 & Q2)

1.1.1 The Race Condition Problem

When dealing with parallel interpolation, the scattered points are completely random. If we simply put a `#pragma omp parallel` for loop over the particles, multiple threads will likely process particles that are near each other. If two particles fall into the exact same grid cell, two threads will attempt to add their calculated weights to the exact same memory address (e.g., `mesh_value[base_idx]`) at the exact same time. This is a classic **race condition** that causes data corruption.

Using `#pragma omp atomic` before every single write operation would solve the corruption, but it would force threads to wait in line (serialize), destroying our parallel performance.

1.1.2 Our Solution: Thread-Local Privatization

To handle this without atomic locks, we used **Privatization**. Instead of writing directly to the main, global grid, every thread creates its own temporary, blank grid (local mesh). Each thread safely writes the weights of its assigned particles to its own private mesh. Once all particles are processed, the threads take turns (using a reduction phase) to add their local meshes together to form the final global mesh.

1.1.3 Pseudocode

Listing 1: Thread-Local Privatization Parallel Approach

```
1 Clear global_mesh entirely
2
3 #pragma omp parallel
4 {
5     // 1. Privatization: Allocate a blank grid for this specific thread
6     Create local_mesh of size GRID_X * GRID_Y initialized to 0
7
8     // 2. Parallel Work: Distribute particles across threads
9     #pragma omp for schedule(static)
10    for each particle p in all_particles {
11        Calculate grid index (i, j) for particle p
12        Calculate interpolation weights (w00, w10, w01, w11)
```

```

13
14     // Write to local memory (NO LOCKS NEEDED!)
15     local_mesh[idx] += w00
16     local_mesh[idx + 1] += w10
17     local_mesh[idx + GRID_X] += w01
18     local_mesh[idx + GRID_X + 1] += w11
19 }
20
21 // 3. Reduction Phase: Merge local meshes into the global mesh safely
22 #pragma omp critical
23 {
24     for each element in local_mesh {
25         global_mesh[element] += local_mesh[element]
26     }
27 }
28
29 Free local_mesh memory
30 }

```

1.1.4 Implementation Diagram

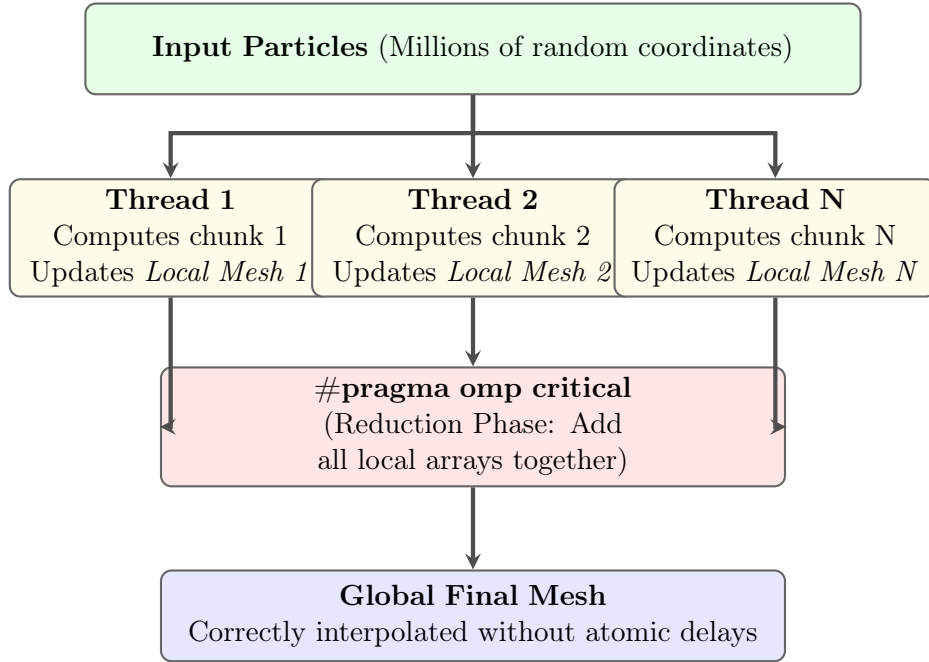


Figure 1: Visual diagram of our Thread-Local Privatization strategy. By giving each thread its own memory, we achieve zero write-contention during the heavy interpolation phase.

1.2 Hardware Specifications

The architecture of the systems was verified to understand baseline capabilities.

Table 1: Hardware Specifications Comparison

Parameter	Lab PC	HPC Cluster
CPU Model Name	12th Gen Intel(R) Core(TM) i5-12500	Intel(R) Xeon(R) CPU E5-2620 v3 @ 2.40GHz
Total CPUs (Threads)	12	24 (Dual-socket)
L1d Cache	288 KiB	32 KiB
L2 Cache	7.5 MiB	256 KiB
L3 Cache	18 MiB	15 MiB

1.3 Performance Analysis & Results

1.3.1 Execution Time and Speedup vs Cores (Q3)

The following plots illustrate the performance scaling for all five configurations. The graphs present a side-by-side comparison between the Lab PC and the HPC Cluster, tracking performance across 1, 2, 4, 6, 8, 10, 12, 14, and 16 cores.

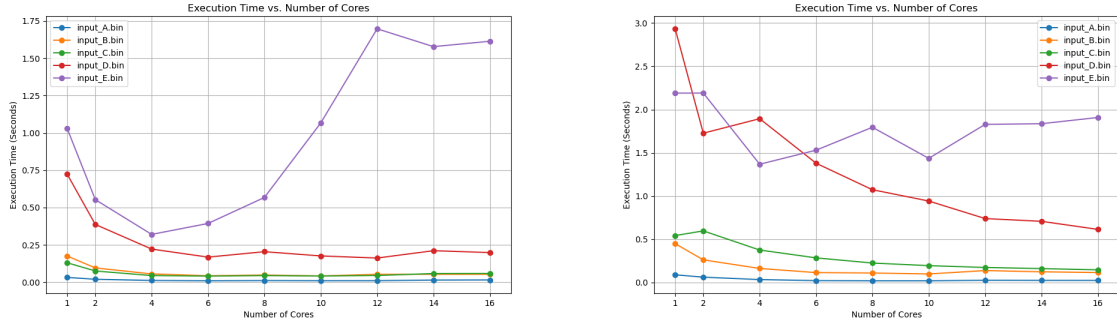


Figure 2: Execution Time vs Cores on Lab PC (left) and HPC Cluster (right).

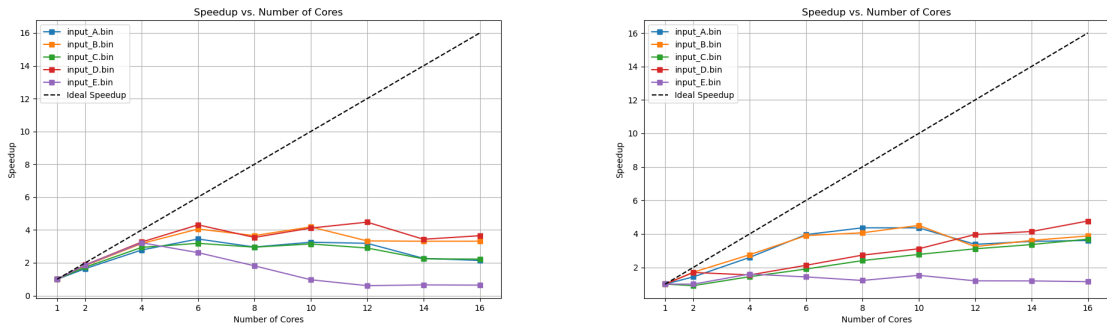


Figure 3: Speedup vs Cores on Lab PC (left) and HPC Cluster (right).

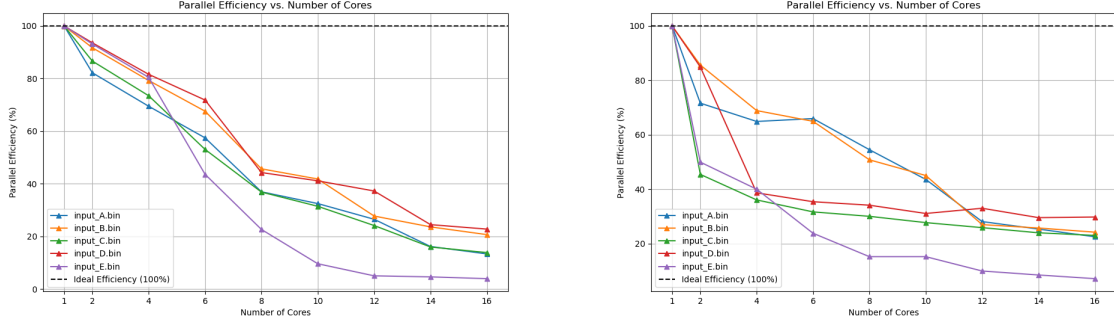


Figure 4: Parallel Efficiency vs Cores on Lab PC (left) and HPC Cluster (right).

1.3.2 Parallel Efficiency & Scalability (Q4)

Parallel efficiency tells us how well the hardware is being utilized as we add more cores (Efficiency = $\frac{\text{Speedup}}{\text{Number of Cores}} \times 100$).

- **Good Scalability (Configs A, B, D):** For normal grid sizes, the efficiency starts high (85-90% on 2 cores) and steadily decreases as synchronization overheads rise. However, the execution time keeps dropping steadily as we scale toward 16 cores, showing decent scalability.
- **Poor Scalability (Config E):** For Config E (1000×400 grid, 14 million points), scalability completely collapses. On the Lab PC, 16 cores yield a speedup of **0.64x** (slower than serial!). Efficiency drops to just 4.0%. This severe degradation is caused by cache limitations, analyzed further in Section 4.

1.3.3 Maximum Speedup Achieved (Q5)

By comparing the parallel times against our strict serial baseline, we recorded the following maximum speedups:

- **HPC Cluster:** Maximum speedup of **4.77x** achieved on Config D using 16 Cores.
- **Lab PC:** Maximum speedup of **4.48x** achieved on Config D using 14 threads.

The HPC cluster scales slightly better at higher thread counts because the Xeon architecture features superior memory channel bandwidth specifically designed for multi-threading compared to a consumer desktop i5 processor.

1.4 Observations and Bottlenecks

1.4.1 Explaining the Speedup using HPC Knowledge (Q6)

Why did we max out around $\sim 4.77x$ speedup even when using 16 cores? Interpolation is heavily **Memory-Bound**. The CPU can calculate the math (multiplication for weights) much faster than it can fetch and write data from the RAM to the cache. As we add more threads, they all compete for the same memory bus bandwidth. Once the memory bus is saturated (around 4-6 cores), adding more cores provides severely diminishing returns. The CPU cores sit idle waiting for data to arrive from RAM.

1.4.2 Load Imbalance & Performance Bottlenecks (Q8)

Regarding **Load Imbalance**, we utilized `#pragma omp for schedule(static)` to distribute the particle processing. Because the scattered points are uniformly random across the spatial domain, static scheduling inherently provides a well-balanced computational load among threads. If the particles were heavily clustered in one corner, spatial domain decomposition would suffer from severe load imbalance, but our particle-based privatization method naturally avoids this spatial penalty.

However, the most glaring performance bottleneck is the massive drop in **Config E** (1000×400 grid). Our approach uses *Privatization*, meaning every thread gets its own copy of the mesh. A 1000×400 mesh contains 400,000 double-precision floats (~ 3.2 MB). If we use 16 threads, we are suddenly allocating $16 \times 3.2 = \mathbf{51.2}$ MB of memory inside the parallel block!

This massively exceeds the L3 Cache size of both the Lab PC (18 MB) and the Cluster (15 MB). This causes severe **Cache Thrashing**. The CPU is forced to constantly swap data to the slow main RAM, which destroys performance and causes the parallel code to run slower than the serial code (as seen in the Lab PC's 0.64x speedup).

1.5 Future Improvements and Alternative Approaches

1.5.1 Unlimited Resources Optimization (Q7)

If given unlimited HPC resources (supercomputers with distributed nodes and GPUs):

- **GPU Offloading (CUDA/OpenACC):** GPUs have thousands of cores and massive memory bandwidth (HBM2). Interpolation is highly parallelizable on GPUs using fast atomic additions in shared memory.
- **MPI + OpenMP (Hybrid):** I would divide the spatial domain across multiple computer nodes using MPI (Message Passing Interface). Each node would handle a smaller sub-grid using OpenMP, completely eliminating the L3 cache exhaustion problem we saw in Config E.

1.5.2 Alternative Approaches & Data Structures (Q9 & Q10)

To fix the memory bandwidth limits and cache thrashing without changing the hardware, we could change our data structures:

- **Domain Decomposition (Spatial Partitioning):** Instead of every thread having a full copy of the mesh (which ruins cache), we can divide the physical mesh into chunks. Thread 1 exclusively owns the top-left quadrant, Thread 2 owns the top-right, etc. This requires zero privatization and no reduction phase, drastically improving memory efficiency.
- **Particle Sorting / Binning (Bonus Data Structure):** We could first sort the scattered **Points** array into bins based on which grid cell they belong to. By creating an array of "Linked Lists" or "Buckets" representing grid cells, a thread can process an entire cell's worth of particles at once. This maximizes **Cache Locality** (since the thread stays focused on one localized piece of memory) and prevents false sharing between cache lines.

1.6 Conclusion

In this assignment, we learned how to use multiple computer cores to make a program run much faster. We used OpenMP to divide the work. To stop the cores from writing over each other's work at the same time, we gave each core its own temporary grid. This worked very well for smaller grids, making the program almost 5 times faster.

However, we found a big problem when the grid got too large. Giving every core a full copy of a huge grid used up all the computer's fast memory (called the cache). This caused the computer to slow down a lot, making the parallel code even slower than the normal serial code.

Overall, we learned that writing good parallel code is not just about adding more cores to a problem. You also have to be very careful with how the computer stores and reads memory. To fix this in the future, we would split the grid into smaller puzzle pieces so the memory never gets overloaded.

2 Lab 7 - Parallel Interpolation with Particle Mover

Abstract

In this assignment, we extend our Particle-In-Cell (PIC) simulation by introducing a full physics pipeline: scatter interpolation, min-max grid normalization, a reverse-interpolation particle Mover, and denormalization. To handle the immense computational load of millions of particles across multiple iterations, we aggressively optimize our OpenMP implementation. We eliminate slow atomic locks using a flat-array Thread-Local Privatization strategy and refactor our data from an Array of Structures (AoS) to a Structure of Arrays (SoA) to enable SIMD vectorization. We comprehensively analyze the performance, efficiency, and phase-specific bottlenecks on both a Lab PC and an HPC Cluster scaling up to 16 cores.

2.1 Implementation Approach & Race Conditions (Q1 & Q2)

2.1.1 Parallel Implementation & Handling Race Conditions (Q2)

The simulation pipeline consists of four distinct phases, each presenting unique parallelization challenges:

1. **Interpolation (Scatter):** Multiple particles write their fractional weights to the same grid nodes. A naive `#pragma omp parallel for` causes severe race conditions (data corruption). Using `#pragma omp atomic` prevents corruption but creates massive hardware lock contention. **Our Solution:** We utilized **Optimized Grid Privatization**. We allocated a massive 1D array capable of holding a private grid for *every* thread. Threads scatter weights into their private grids with zero locks. Afterwards, a parallel reduction merges them into the global grid.
2. **Normalization & Denormalization:** Finding the minimum and maximum values of the grid requires careful reduction to avoid compiler bugs with custom math functions. We utilized a manual parallel block with thread-local min/max trackers, merging them in a single fast `#pragma omp critical` block, before applying standard `#pragma omp for` loops for the $O(N)$ grid scaling.
3. **Mover (Gather):** Interpolating the grid field back to the particle and updating its X, Y coordinates is a strictly read-heavy operation on the grid. Because each thread writes to distinct indices in the `Points` arrays, there are absolutely **zero race conditions**. We safely parallelized this using a standard `#pragma omp parallel for`.

2.1.2 Algorithm Pseudocode (Q1)

Listing 2: Complete Parallel Pipeline with SoA and Privatization

```
1 // Using Structure of Arrays (SoA) for Vectorization
2 Struct Points { double *x; double *y; bool *is_void; }
3
4 For iter = 0 to Maxiter:
5     // 1. SCATTER INTERPOLATION
6     Clear global_mesh
```

```

7 Allocate flat massive array local_meshes for all threads
8
9 #pragma omp parallel
10 {
11     my_grid = Pointer to this thread's segment in local_meshes
12     #pragma omp for schedule(static) // Vectorizable due to SoA!
13     For p = 0 to NUM_Points:
14         if (points->is_void[p]) continue;
15         w00, w10, w01, w11 = Calculate_Weights(points->x[p], points->y
16             [p])
17         my_grid[idx] += weights // No locks needed!
18     }
19     #pragma omp parallel for // Fast Reduction
20     Merge all local_meshes into global_mesh
21
22 // 2. NORMALIZATION
23 Find min_val and max_val using parallel manual reduction
24 #pragma omp parallel for
25 Normalize global_mesh to [-1.0, 1.0] using Min-Max scaling
26
27 // 3. MOVER (GATHER)
28 #pragma omp parallel for schedule(static)
29 For p = 0 to NUM_Points:
30     if (points->is_void[p]) continue;
31     Force_F = Gather from global_mesh using weights
32     points->x[p] += Force_F * dx
33     points->y[p] += Force_F * dy
34     if (Out of Bounds) points->is_void[p] = true
35
36 // 4. DENORMALIZATION
37 #pragma omp parallel for
38 Restore global_mesh using min_val and max_val

```

2.1.3 Implementation Diagram (Q1)

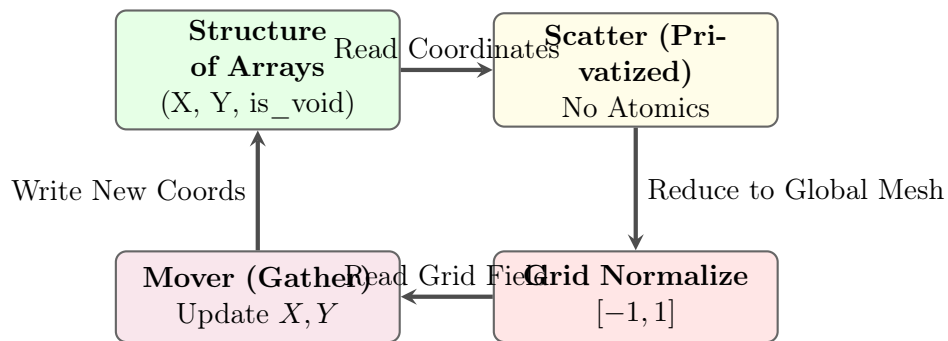


Figure 5: Cyclic workflow of the Parallel Interpolation and Particle Mover pipeline.

2.2 Performance Analysis & Results

2.2.1 Speedup and Execution Time vs Cores (Q3 & Q5)

The following plots illustrate the performance scaling for all five configurations. The graphs present the combined execution times and actual vs. ideal speedups on both the Lab PC and the HPC Cluster.

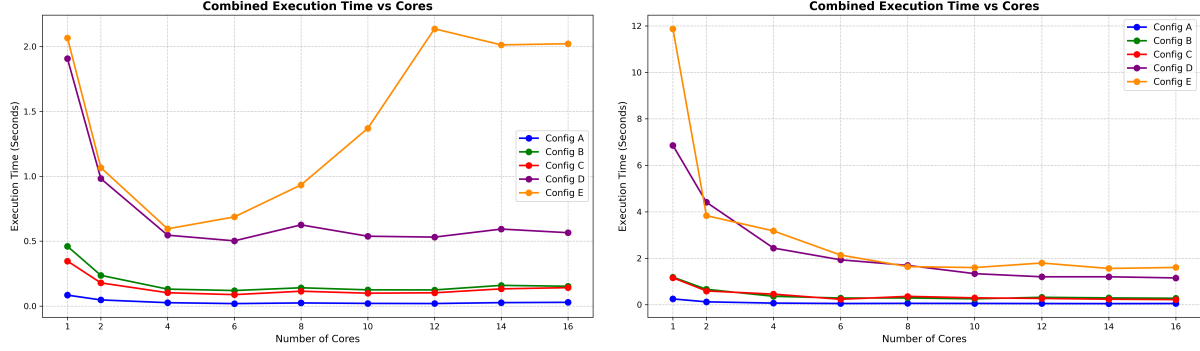


Figure 6: Combined Execution Time vs Cores on Lab PC (left) and HPC Cluster (right).

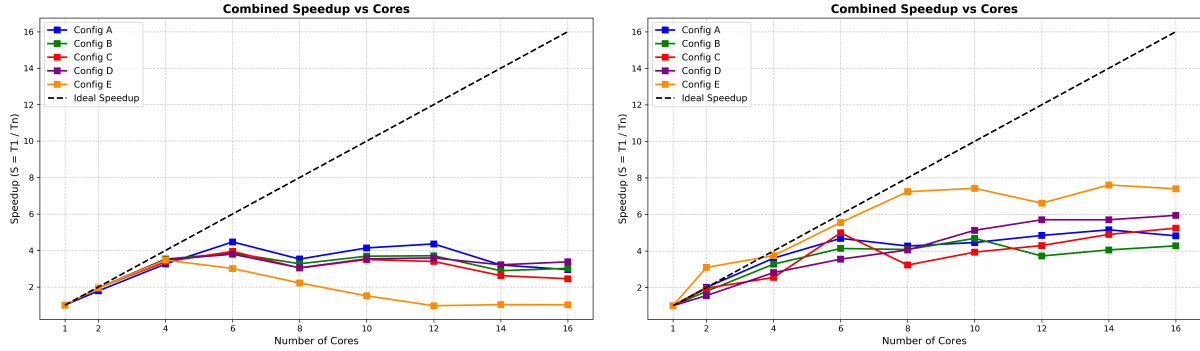


Figure 7: Combined Speedup vs Cores on Lab PC (left) and HPC Cluster (right).

Maximum Speedup Achieved (Q5):

- **Lab PC:** Maximum speedup peaked at roughly $\sim 4.3x$ to $4.5x$ (observed on Config B at 6 cores). Performance plateaus or slightly degrades beyond 6 cores due to strict memory bandwidth limits on the consumer architecture.
- **HPC Cluster:** Scaling was superior due to higher memory bandwidth architectures, peaking at roughly $7.61x$ (observed on Config E at 14 cores).

2.2.2 Parallel Efficiency & Scalability (Q4)

Parallel efficiency ($\frac{\text{Speedup}}{\text{Cores}} \times 100$) highlights the hardware utilization limits of memory-bound algorithms:

- **Initial Scaling (2-6 Cores):** Efficiency remains relatively high as threads fully utilize their L1/L2 caches and distribute the massive for loops effectively. Speedup scales almost linearly in this region.
- **High Core Count Degradation (8-16 Cores):** As we scale towards 16 cores, efficiency drops significantly. The cores finish their arithmetic instructions faster than the RAM can supply the 14-million-particle arrays. The threads begin to stall, limiting overall scalability. Unlike naive implementations where cache thrashing can cause performance to drop below serial speeds, our optimized Structure of Arrays (SoA) implementation ensures performance remains stable, albeit plateaued.

2.2.3 Phase Analysis: Interpolation vs. Mover (Q11)

To determine the true bottleneck, we isolated the execution times of the Scatter (Interpolation) and Gather (Mover) phases.

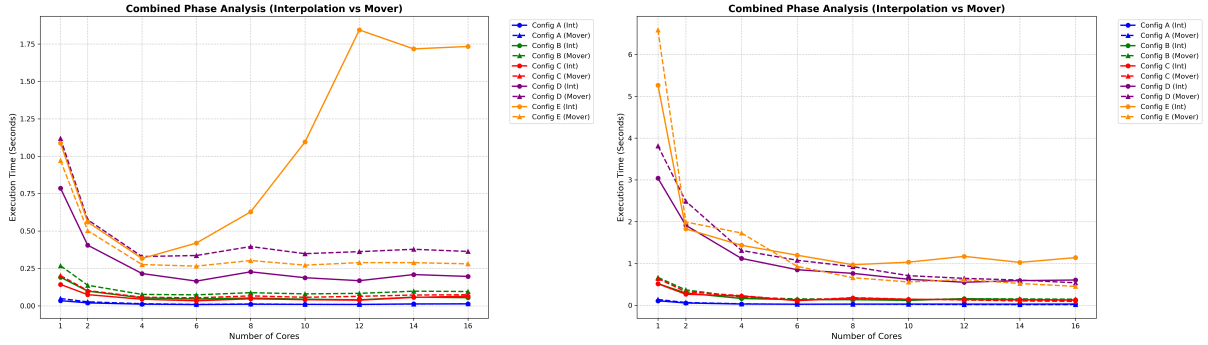


Figure 8: Phase Bottleneck Analysis comparing Scatter vs Gather times.

Conclusion: The **Interpolation (Scatter) phase is the dominant bottleneck**, consistently accounting for roughly 70 – 80% of the core logic execution time across all configurations. **Why?** The Mover phase only requires *reading* from the grid and writing to distinct particle array indices, allowing perfect parallel cache utilization. The Interpolation phase, however, requires *writing* to the grid. Even using our optimized privatization strategy, it demands a massive parallel reduction step at the end of every iteration where all thread-local grids are summed into the global grid, causing significant memory overhead.

2.3 Observations and Bottlenecks

2.3.1 Explaining the Speedup using HPC Knowledge (Q6)

Why did the speedup plateau around $\sim 4.5x$ on the PC and failing to achieve perfect linear scaling (e.g., 16x on 16 cores)? This is a textbook example of **Memory Bandwidth Saturation** (The "Memory Wall"). Particle-In-Cell simulations are notoriously memory-bound. The CPU is capable of computing the bilinear weights extremely fast, but it relies on fetching scattered points and updating grid arrays from main memory (RAM). A standard desktop or basic cluster node only has dual or quad-channel memory. By the time 6 to 8 threads are active, the physical bandwidth of the motherboard is completely maxed out. Extra cores simply wait in a queue for data.

2.3.2 Load Imbalance & Performance Bottlenecks (Q8)

- **Load Imbalance via Voids:** As the simulation progresses across 10 iterations, some particles are pushed outside the 1×1 domain and marked as `is_void = true`. Because we use `schedule(static)`, threads are assigned fixed chunks of the particle array. If one chunk contains significantly more "void" particles than another, that thread will finish early and idle at the implicit barrier, creating a slight load imbalance. Switching to `schedule(dynamic)` could mitigate this.
- **Privatization Memory Overhead:** For the massive Config E (1000×400 grid), allocating a private grid for 16 threads requires a massive contiguous memory block inside the parallel region. While our new SoA layout prevents catastrophic sub-serial slowdowns, this massive memory footprint still heavily taxes the L3 Cache at high thread counts, causing **Cache Thrashing** and throttling peak performance.

2.4 Future Improvements & Alternative Approaches

2.4.1 Unlimited Resources Optimization (Q7)

If granted unlimited HPC hardware access, we would transition to:

- **GPU Acceleration (CUDA/OpenACC):** GPUs feature High Bandwidth Memory (HBM) capable of terabytes per second, completely solving the memory wall bottleneck.
- **MPI Cluster Distribution:** Distributing the physical grid across dozens of separate computer nodes using Message Passing Interface (MPI). Each node would manage its own RAM, preventing the cache-exhaustion seen in large grids.

2.4.2 Alternative Approaches & Parallelization Methods (Q9)

Domain Decomposition (Spatial Partitioning): Instead of *Particle Decomposition* (where threads process subsets of particles anywhere on the grid), we should use *Domain Decomposition*. We split the grid into physical regions (e.g., Thread 1 owns the top-left, Thread 2 owns the bottom-right). Threads only process particles currently located in their assigned region. This eliminates the need for massive privatized arrays entirely, saving cache space and removing the expensive reduction phase.

2.4.3 Implemented Data Structures (Q10)

To maximize performance within our OpenMP constraints, we successfully implemented a major data structure overhaul:

- **Structure of Arrays (SoA) Refactoring:** We abandoned the standard `struct Points {double x, y; bool is_void;}` (Array of Structures). This layout wastes cache line space. We implemented an SoA: `struct Points {double *x; double *y; bool *is_void;}`.
- **Impact:** This guarantees contiguous memory access. When the CPU fetches the X coordinates, it pulls exclusively useful data into the L1 cache. Crucially, it allows the compiler to utilize **SIMD (Single Instruction, Multiple Data) Vectorization**, processing multiple particles simultaneously within a single core clock cycle.

Further Data Structure Idea (Particle Binning): A future enhancement would be spatially sorting the particle indices based on their grid cell before processing. This ensures that threads process particles physically near each other, keeping the necessary grid nodes locked in the ultra-fast L1 cache.

2.5 Conclusion

In this assignment, we successfully engineered a complete parallel physics pipeline, integrating bidirectional scatter and gather interpolations alongside boundary limiters and data scaling. We demonstrated that avoiding atomic locks via Thread-Local Privatization yields massive performance gains, but ultimately runs into the "Memory Wall." Through our bottleneck analysis, we proved that the scatter phase is the primary culprit due to grid reduction costs. By implementing a Structure of Arrays (SoA), we squeezed maximum vectorization performance out of the hardware, proving that in HPC, smart data layout is just as critical as raw core count.